Pipeline CI/CD pour BobApp

Documentation

Auteur : Julio Daniel GIL CANO.
Développeur full-stack

21/03/2026

Table des matières

1 Contexte 3
2 Problématique 3
3 Objectifs du projet 3
4 Outils utilisés 4
4.1 GitHub Actions 4
4.2 SonarCloud 4
4.3 Maven 4
4.4 JaCoCo 4
4.5 Angular et Karma + Jasmine 4
4.6 Docker 5
5 Fonctionnement du pipeline CI/CD 5
5.1 Workflow CI Backend (ci-backend.yml) 7
5.2 Workflow CI Frontend (ci-frontend.yml) 9
5.3 Workflow CD (cd.yml) 11
6 Analyse des métriques SonarCloud 15
6.1 Coverage 15
6.2 Reliability (Bugs) 16
6.3 Maintainability 16
6.4 Security 17
6.5 Duplications 18
6.6 Hotspots Reviewed 18
7 Choix des KPI 19
8 Configuration des KPI dans SonarCloud 19
9 Analyse des avis utilisateurs 19
9.1 Problème 1 : bug sur la publication de blagues 19
9.2 Problème 2 : correction des bugs trop lente 19
9.3 Problème 3 : absence de réponse aux utilisateurs 19
10 Conclusion 20

1 Contexte

BobApp est une application web permettant aux utilisateurs de consulter et partager des blagues.
L'application est composée de deux parties principales :

- *un backend développé en Java avec Spring Boot*
- *un frontend développé en Angular*

Le projet est actuellement développé par Bob sur son temps libre.
Avec l'augmentation du nombre d'utilisateurs, plusieurs problèmes sont apparus, notamment concernant la qualité du code et la gestion des déploiements.

Actuellement, Bob doit effectuer manuellement plusieurs étapes lors du développement :

- *validation des Pull Requests*
- *exécution des tests*
- *compilation de l'application*
- *vérification du bon fonctionnement*
- *déploiement manuel via FTP*

Ce processus est long, répétitif et sujet aux erreurs. De plus, il ralentit fortement le développement de nouvelles fonctionnalités.

2 Problématique

Plusieurs utilisateurs ont récemment publié des avis négatifs concernant BobApp.

Voici quelques exemples de retours utilisateurs :

Note	***Avis utilisateur***
⭐ 1/5	Impossible de poster une suggestion de blague, le bouton fait planter le navigateur
⭐ 2/5	Un bug signalé depuis deux semaines n'est toujours pas corrigé
⭐ 1/5	Je ne reçois plus rien et je n'ai aucune réponse du support
⭐ 2/5	J'ai supprimé ce site de mes favoris

Ces avis mettent en évidence plusieurs problèmes :

- *présence de bugs non corrigés*
- *manque de visibilité sur la qualité du code*
- *temps de correction trop long*
- *processus de déploiement inefficace*

Afin d'améliorer la qualité du projet et d'automatiser les tâches répétitives, la mise en place d'un pipeline CI/CD est nécessaire.

3 Objectifs du projet

L'objectif principal de ce projet est de mettre en place un pipeline CI/CD complet afin d'améliorer la qualité et la maintenance de l'application.

Les objectifs sont les suivants :

- *automatiser l'exécution des tests*

- *analyser la qualité du code*
- *générer automatiquement les rapports de couverture*
- *construire les images Docker*
- *déployer automatiquement les conteneurs sur DockerHub*

Pour réaliser cela, plusieurs outils d'automatisation ont été utilisés.

4 Outils utilisés

La mise en place du pipeline CI/CD repose sur plusieurs outils.

4.1 GitHub Actions

Le pipeline d'intégration continue est automatisé avec GitHub Actions.
GitHub Actions permet de créer des workflows exécutés automatiquement lors de certains événements:

- *push sur le dépôt*
- *création de Pull Request*

Ces workflows permettent d'automatiser les différentes étapes du pipeline.

4.2 SonarCloud

L'analyse de la qualité du code est réalisée avec SonarCloud.
Cet outil permet d'analyser automatiquement :

- *les bugs*
- *les vulnérabilités*
- *les problèmes de maintenabilité*
- *la couverture des tests*

Les résultats sont visibles directement dans l'interface SonarCloud.

4.3 Maven

Le backend utilise Apache Maven pour :

- *compiler le projet*
- *exécuter les tests*
- *générer les rapports de couverture*

4.4 JaCoCo

La couverture de tests du backend est générée avec JaCoCo.
Le rapport de couverture est produit sous forme d'un fichier XML utilisé par SonarCloud.

4.5 Angular et Karma + Jasmine

Le frontend est développé avec Angular.
Les tests sont exécutés avec :

- *Karma*
- *Jasmine*

Ces outils permettent de générer un rapport de couverture utilisé par SonarCloud.

4.6 Docker

Le déploiement utilise Docker.
Deux images Docker sont générées :

- *une image pour le frontend*
- *une image pour le backend*

Ces images sont ensuite publiées sur DockerHub.

5 Fonctionnement du pipeline CI/CD

Afin d'automatiser les différentes étapes du développement et du déploiement de l'application BobApp, plusieurs workflows GitHub Actions ont été mis en place.

Le pipeline CI/CD repose sur trois workflows principaux :

- *un workflow CI pour le backend*
- *un workflow CI pour le frontend*
- *un workflow CD pour la construction et la publication des images Docker*

*Ces workflows sont définis dans des fichiers YAML situés dans le dossier : **.github/workflows***
Les fichiers utilisés sont les suivants :

Fichier	***Rôle***
ci-backend.yml	Pipeline CI du backend
ci-frontend.yml	Pipeline CI du frontend
cd.yml	Pipeline CD pour le build et le déploiement Docker

La structure générale du projet est organisée de la manière suivante :

```
bobapp
│
├── .github
│   └── workflows
│       ├── ci-backend.yml
│       ├── ci-frontend.yml
│       └── cd.yml
├── back
│   ├── src
│   ├   :
│   ├── pom.xml
│   ├── Dockerfile
│   └── sonar-project.properties
├── front
│   ├── src
│   ├   :
│   ├── package.json
│   ├── Dockerfile
│   └── sonar-project.properties
├── docker-compose.yml
└── README.md
```

Les fichiers sonar-project.properties permettent de configurer l'analyse de qualité du code réalisée par SonarCloud.
Deux fichiers sont utilisés :

Fichier	***Rôle***
back/sonar-project.properties	configuration SonarCloud pour le backend
front/sonar-project.properties	configuration SonarCloud pour le frontend

Ces fichiers définissent notamment :

- *l'identifiant du projet SonarCloud*
- *les dossiers contenant le code source*
- *les dossiers contenant les tests*
- *l'emplacement des rapports de couverture de tests*
- *les exclusions*

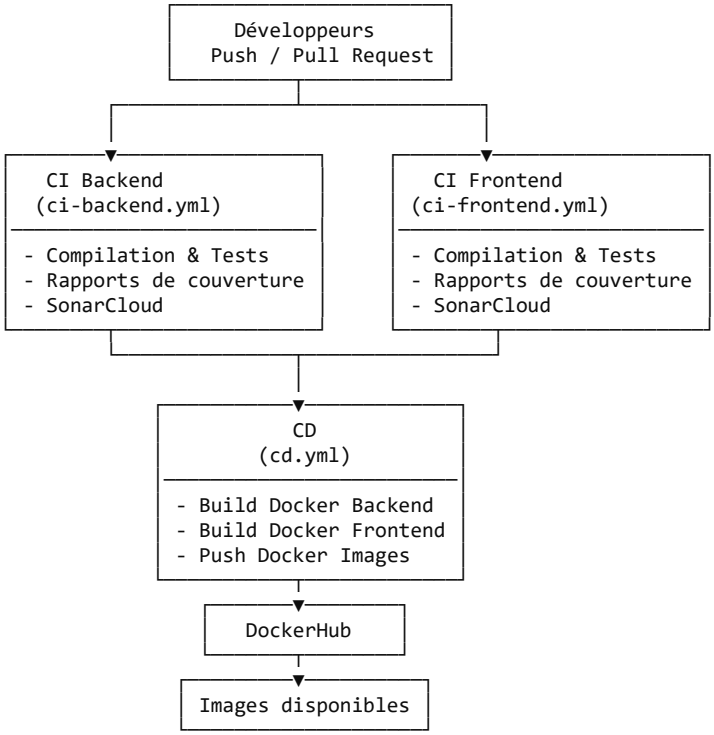
Le pipeline CI/CD repose sur deux types de processus :

- ***Continuous Integration (CI)***
  Les workflows CI ont pour objectif de :
  - *compiler l'application*
  - *exécuter les tests*
  - *générer les rapports de couverture*
  - *analyser la qualité du code avec SonarCloud*

  Deux workflows CI sont utilisés :
  - *ci-backend.yml*
  - *ci-frontend.yml*
- ***Continuous Delivery (CD)***
  Le workflow CD a pour objectif de :
  - *construire les images Docker*
  - *vérifier que le projet peut être compilé*
  - *publier les images Docker sur DockerHub*

  Ce workflow est défini dans le fichier :
  - *cd.yml*

Un schéma global du pipeline CI/CD est présenté ci-dessous.



5.1 Workflow CI Backend (ci-backend.yml)

Ce workflow est dédié au backend (Spring Boot). Il est exécuté automatiquement lors des événements suivants :

```
on:
  push:
    paths:
      - 'back/**'
    branches: [ "feature/*", "release/*", "hotfix/*", "develop" ]
  pull_request:
    paths:
      - 'back/**'
    branches: [ main ]
    types: [ opened, synchronize, reopened ]
```

Ces événements permettent de déclencher le pipeline lorsque :

- *un développeur pousse du code sur une branche de développement*
- *une Pull Request est créée vers la branche main ou mise à jour (ajout de commits)*

Le workflow CI est composé de plusieurs étapes :

```
jobs:
  build-test-sonar:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: back

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 11

      - name: Run tests
        run: mvn clean verify

      - name: Upload JaCoCo report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: back/target/site/jacoco

      - name: SonarCloud Scan
        uses: SonarSource/sonarqube-scan-action@v2
        with:
          projectBaseDir: back
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN_BACK }}
```

Les étapes principales du pipeline sont les suivantes :

a. *Récupération du code*

Code YAML :

```
- uses: actions/checkout@v4
```

On récupère ici le code source du dépôt GitHub.

b. *Installation de l'environnement*

Code YAML :

```
- uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: 11
```

Cette étape installe l'environnement Java nécessaire à la compilation du backend.

c. *Exécution des tests*

Code YAML :

```
- name: Run tests
  run: mvn clean verify
```

Cette commande compile le projet, exécute les tests unitaires et JaCoCo génère automatiquement un rapport de couverture dans : back/target/site/jacoco/jacoco.xml.

d. *Upload du rapport de couverture*

Code YAML :

```
- name: Upload JaCoCo report
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: back/target/site/jacoco
```

Cette action permet de sauvegarder les rapports de couverture dans GitHub Actions.

e. *Analyse du code avec SonarCloud dans la Pull Request*

Code YAML :

```
- name: SonarCloud Scan
  uses: SonarSource/sonarqube-scan-action@v2
  with:
    projectBaseDir: back
  env:
    SONAR_TOKEN: ${{ secrets.SONAR_TOKEN_BACK }}
```

Cette étape :

- *Envoie le code de la Pull Request à SonarCloud.*
- *SonarCloud analyse automatiquement la qualité du code, la couverture de tests, les bugs, les vulnérabilités, les duplications, etc.*
- *Les résultats sont visibles dans l'interface SonarCloud et sont directement associés à la Pull Request dans GitHub.*

f. *Validation via le Quality Gate (blocage de la PR)*

Après l'analyse, SonarCloud applique un Quality Gate, c'est-à-dire un ensemble de règles définissant si le code est acceptable ou non.

Exemples de règles :

- *Coverage ≥ 80 % sur le nouveau code*
- *Aucun nouveau bug*
- *Aucune nouvelle vulnérabilité*

Deux cas sont possibles :

- *Quality Gate validé (SUCCESS)*
  La Pull Request respecte les critères de qualité et peut être fusionnée.
- *Quality Gate échoué (FAILED)*
  La Pull Request ne respecte pas les critères définis.

Dans ce cas :

- *SonarCloud remonte un statut failed dans GitHub*
- *Si une règle de protection de branche (branch protection) est configurée :*
  - ✓ *le merge de la Pull Request est bloqué automatiquement*
  - ✓ *le développeur doit corriger les problèmes avant de pouvoir fusionner*

L'intégration de SonarCloud dans le pipeline CI permet de transformer chaque Pull Request en point de contrôle qualité. Grâce au Quality Gate, seules les modifications respectant les standards définis peuvent être intégrées dans la branche principale, garantissant ainsi un niveau de qualité constant du projet.

5.2 Workflow CI Frontend (ci-frontend.yml)

Ce workflow est dédié au frontend (Angular). Il est exécuté automatiquement lors des événements suivants :

```
on:
  push:
    paths:
      - 'front/**'
    branches: [ "feature/*", "release/*", "hotfix/*", "develop" ]
  pull_request:
    paths:
      - 'front/**'
    branches: [ main ]
    types: [ opened, synchronize, reopened ]
```

Ces événements permettent de déclencher le pipeline lorsque :

- *un développeur pousse du code sur une branche de développement*
- *une Pull Request est créée vers la branche main ou mise à jour (ajout de commits)*

Le workflow CI est composé de plusieurs étapes :

```
jobs:
  build-test-sonar:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: front

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 18

      - run: npm ci

      - name: Run tests
        run: npm run test -- --watch=false --code-coverage

      - name: Upload coverage
        uses: actions/upload-artifact@v4
        with:
          name: front-coverage
          path: front/coverage

      - name: SonarCloud Scan
        uses: SonarSource/sonarqube-scan-action@v2
        with:
          projectBaseDir: front
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN_FRONT }}
```

Les étapes principales du pipeline sont les suivantes :

a. Récupération du code

Code YAML :

```
- uses: actions/checkout@v4
```

Cette étape récupère le code source du dépôt GitHub.

b. Installation de l'environnement

Code YAML :

```
- uses: actions/setup-node@v4
  with:
    node-version: 18

- run: npm ci
```

Cette étape installe l'environnement et les dépendances nécessaires à la compilation du frontend.

c. Exécution des tests

Code YAML :

```
- name: Run tests
  run: npm run test -- --watch=false --code-coverage
```

Cette commande compile le projet, exécute les tests unitaires avec Karma + Jasmine et génère automatiquement un rapport de couverture dans : front/coverage/lcov.info.

d. Upload du rapport de couverture

Code YAML :

```
- name: Upload coverage
  uses: actions/upload-artifact@v4
  with:
    name: front-coverage
    path: front/coverage
```

Cette étape consiste à sauvegarder les rapports de couverture dans GitHub Actions.

e. Analyse du code avec SonarCloud dans la Pull Request (PR)

Code YAML :

```
- name: SonarCloud Scan
  uses: SonarSource/sonarqube-scan-action@v2
  with:
    projectBaseDir: front
  env:
    SONAR_TOKEN: ${{ secrets.SONAR_TOKEN_FRONT }}
```

Cette étape :

- *Envoie le code de la Pull Request à SonarCloud.*
- *SonarCloud analyse automatiquement la qualité du code, la couverture de tests, les bugs, les vulnérabilités, les duplications, etc.*
- *Les résultats sont visibles dans l'interface SonarCloud et sont directement associés à la Pull Request dans GitHub.*

f. Validation via le Quality Gate (blocage de la Pull Request)

Après l'analyse, SonarCloud applique un Quality Gate, c'est-à-dire un ensemble de règles permettant de valider la qualité du code.

Deux cas sont possibles :

- *Quality Gate validé (SUCCESS)*
  La Pull Request respecte les critères de qualité et peut être fusionnée.

- *Quality Gate échoué (FAILED)*
  La Pull Request ne respecte pas les critères définis.

Dans ce cas :

- *SonarCloud remonte un statut failed dans GitHub*
- *Si une règle de protection de branche (branch protection) est configurée :*
  - ✓ *le merge de la Pull Request est bloqué automatiquement*
  - ✓ *le développeur doit corriger les problèmes avant de pouvoir fusionner*

Cette intégration garantit que le frontend respecte le même niveau d'exigence qualité que le backend, assurant une cohérence globale du projet.

5.3 Workflow CD (cd.yml)

Le workflow CD permet de construire et publier les images Docker. Il est exécuté automatiquement lors des événements suivants :

```
on:
  pull_request:
    branches: [ main ]
    types: [ opened, synchronize, reopened ]
  push:
    branches: [ main ]
```

Ces événements permettent de déclencher le pipeline lorsque :

- *un développeur pousse du code sur la branche principale (main)*
- *une Pull Request est créée vers la branche main ou mise à jour (ajout de commits)*

Le workflow utilise également la configuration suivante :

```
concurrency:
  group: cd-main
  cancel-in-progress: true
```

Cette configuration permet de gérer l'exécution concurrente des workflows :

- *group: cd-main*
  définit un groupe de workflows liés à la branche principale. Tous les workflows appartenant à ce groupe sont considérés comme mutuellement exclusifs.
- *cancel-in-progress: true*
  annule automatiquement les exécutions précédentes du workflow si un nouveau pipeline est déclenché.

Cela permet d'éviter :

- *l'exécution inutile de plusieurs pipelines en parallèle*
- *les conflits ou incohérences lors du build ou du déploiement*
- *une consommation excessive des ressources*

Par exemple, si plusieurs commits sont poussés rapidement sur main, seul le dernier pipeline sera exécuté.

Le workflow CD est composé de trois jobs :

a. build-back

Code YAML :

```
build-back:
  runs-on: ubuntu-latest
```

```
    if: github.event_name == 'pull_request' || github.event_name == 'push'
    defaults:
      run:
        working-directory: back
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - name: Build Backend Docker Image
        uses: docker/build-push-action@v5
        with:
          context: ./back
          push: false
          tags: juliodanhub57/bobapp-back:latest
```

Ce job sert à construire l'image Docker du backend à partir du dossier back. Il permet de :

- *vérifier que le backend compile correctement*
- *valider le bon fonctionnement du Dockerfile*
- *détecter les erreurs de build avant le déploiement*

Il est exécuté lors des Pull Requests et des push afin de garantir la stabilité du projet.

Les étapes principales du pipeline sont les suivantes :

- ***Récupération du code***
  Code YAML : `- uses: actions/checkout@v4`
  Cette étape récupère le code source du dépôt GitHub afin de permettre son utilisation dans le pipeline.
- ***Configuration de Docker Buildx***
  Code YAML : `- uses: docker/setup-buildx-action@v3`
  Cette étape configure Docker Buildx, un outil avancé permettant de construire des images Docker. Il permet notamment :
  - ✓ *d'améliorer les performances de build*
  - ✓ *de gérer des builds multi-plateformes*
  - ✓ *d'utiliser des fonctionnalités avancées de Docker*
- ***Construction de l'image Docker***
  Code YAML :
  ```
  - name: Build Backend Docker Image
    uses: docker/build-push-action@v5
    with:
      context: ./back
      push: false
      tags: juliodanhub57/bobapp-back:latest
  ```
  Cette étape permet de construire l'image Docker du backend à partir du Dockerfile présent dans le dossier back. Les paramètres utilisés sont :
  - ✓ *context: ./back*
    définit le dossier contenant le code source et le Dockerfile
  - ✓ *push: false*
    indique que l'image est uniquement construite localement et n'est pas envoyée sur DockerHub
  - ✓ *tags: juliodanhub57/bobapp-back:latest*
    définit le nom et la version de l'image Docker

b. *build-front*

Code YAML :

```
build-front:
```

```
runs-on: ubuntu-latest
if: github.event_name == 'pull_request' || github.event_name == 'push'
defaults:
  run:
    working-directory: front
steps:
  - uses: actions/checkout@v4
  - uses: docker/setup-buildx-action@v3
  - name: Build Frontend Docker Image
    uses: docker/build-push-action@v5
    with:
      context: ./front
      push: false
      tags: juliodanhub57/bobapp-front:latest
```

Ce job permet de construire l'image Docker du frontend à partir du dossier front. Il permet de :

- *vérifier que le frontend compile correctement*
- *valider le bon fonctionnement du Dockerfile*
- *détecter les erreurs de build avant le déploiement*

Il est exécuté lors des Pull Requests et des push afin de garantir la stabilité du projet.

Les étapes principales du pipeline sont les suivantes :

- ***Récupération du code***

  Code YAML :

  ```
  - uses: actions/checkout@v4
  ```

  Cette étape récupère le code source du dépôt GitHub afin de permettre son utilisation dans le pipeline.

- ***Configuration de Docker Buildx***

  Code YAML :

  ```
  - uses: docker/setup-buildx-action@v3
  ```

  Cette étape configure Docker Buildx, un outil avancé permettant de construire des images Docker. Il permet notamment :

  - ✓ *d'améliorer les performances de build*
  - ✓ *de gérer des builds multi-plateformes*
  - ✓ *d'utiliser des fonctionnalités avancées de Docker*

- ***Construction de l'image Docker***

  Code YAML :

  ```
  - name: Build Frontend Docker Image
    uses: docker/build-push-action@v5
    with:
      context: ./front
      push: false
      tags: juliodanhub57/bobapp-front:latest
  ```

  Cette étape permet de construire l'image Docker du frontend à partir du Dockerfile présent dans le dossier front. Les paramètres utilisés sont :

  - ✓ *context: ./front*
    définit le dossier contenant le code source et le Dockerfile
  - ✓ *push: false*
    indique que l'image est uniquement construite localement et n'est pas envoyée sur DockerHub
  - ✓ *tags: juliodanhub57/bobapp-front:latest*
    définit le nom et la version de l'image Docker

c. *push-docker*

Code YAML :

```
push-docker:
  needs: [build-back, build-front]
  runs-on: ubuntu-latest
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}
    - name: Push Backend Image
      uses: docker/build-push-action@v5
      with:
        context: ./back
        push: true
        tags: juliodanhub57/bobapp-back:latest
    - name: Push Frontend Image
      uses: docker/build-push-action@v5
      with:
        context: ./front
        push: true
        tags: juliodanhub57/bobapp-front:latest
```

Ce job assure la publication des images Docker du backend et du frontend sur DockerHub.

Ce job :

- *dépend de l'exécution réussie des jobs build-back et build-front :*
  `needs: [build-back, build-front]`
- *s'exécute uniquement après un merge sur la branche main, et jamais lors d'une Pull Request. grâce à la commande :*
  `if: github.event_name == 'push' && github.ref == 'refs/heads/main'`

L'objectif du job est :

- *automatiser le déploiement des images Docker*
- *garantir que seules les versions validées sont publiées*
- *centraliser les images sur DockerHub pour leur utilisation*

Les étapes principales du pipeline sont les suivantes :

- ***Récupération du code***

  Code YAML :

  ```
  - uses: actions/checkout@v4
  ```

  Cette étape récupère le code source du dépôt GitHub afin de permettre son utilisation dans le pipeline.

- ***Authentification à DockerHub***

  Code YAML :

  ```
  - uses: docker/login-action@v3
    with:
      username: ${{ secrets.DOCKERHUB_USERNAME }}
      password: ${{ secrets.DOCKERHUB_TOKEN }}
  ```

  Cette étape permet de se connecter à DockerHub de manière sécurisée.

  Les identifiants sont stockés dans les secrets GitHub, ce qui permet de :

  - ✓ *protéger les informations sensibles*
  - ✓ *éviter de les exposer dans le code*

- ***Construction et publication de l'image backend***

  Code YAML :

```
      - name: Push Backend Image
        uses: docker/build-push-action@v5
        with:
          context: ./back
          push: true
          tags: juliodanhub57/bobapp-back:latest
```

Cette étape :

- ✓ *construit l'image Docker du backend*
- ✓ *publie l'image sur DockerHub grâce à :* `push: true`

- ***Construction et publication de l'image frontend***

  Code YAML :

```
      - name: Push Frontend Image
        uses: docker/build-push-action@v5
        with:
          context: ./front
          push: true
          tags: juliodanhub57/bobapp-front:latest
```

  Cette étape :

  - ✓ *construit l'image Docker du frontend*
  - ✓ *publie l'image sur DockerHub grâce à :* `push: true`

  Ce job constitue la dernière étape du pipeline CI/CD et permet d'automatiser le processus de livraison de l'application en garantissant que seules les versions validées sont déployées.

6 Analyse des métriques SonarCloud

L'analyse du code avec SonarCloud permet d'obtenir plusieurs métriques importantes. L'image ci-dessous présente l'interface SonarCloud avec les métriques.

Ces métriques ont été obtenues après exécution complète du pipeline CI/CD, confirmant le bon fonctionnement de l'intégration continue et de l'analyse SonarCloud.

6.1 Coverage

La métrique Coverage mesure le pourcentage de code exécuté par les tests automatisés. Elle permet d'évaluer le niveau de test du projet et de réduire les risques de bugs lors des modifications du code.

Dans le cas de BobApp, les résultats sont :

Projet	Couverture
Backend (BobApp_Back)	38.8 %
Frontend (BobApp_Front)	80 %

Dans notre projet, le frontend atteint une couverture de 80 %, tandis que le backend présente une couverture de 38.8 %, ce qui reste insuffisant. Cela indique que de nombreuses portions critiques du code ne sont pas encore testées, augmentant le risque de régression.

6.2 Reliability (Bugs)

Cette métrique représente les erreurs potentielles dans le code.

Dans le cas de BobApp, les résultats sont :

Projet	Reliability
Backend (BobApp_Back)	D - 1 bug détecté
Frontend (BobApp_Front)	A – 0 bug détecté

Pour le backend, un bug important a été détecté, ce qui indique un risque élevé de comportement incorrect de l'application dans certaines situations.
Côté frontend, l'analyse n'a trouvé aucun problème de fiabilité.

Un tableau avec un exemple d'interprétation plus général de la métrique ***Reliability*** *est présenté ci-dessous :*

Reliability	Nombre de bugs	Signification
A	0	Aucun bug détecté
B	quelques bugs mineurs	Risque faible
C	plusieurs bugs	Risque modéré
D	bugs importants	Risque élevé
E	bugs critiques	Risque très élevé

6.3 Maintainability

La maintenabilité mesure la facilité de modification du code. Les problèmes détectés sont appelés code smells.
Un code smell est une mauvaise pratique dans le code qui :

- *ne provoque pas forcément un bug*
- *mais rend le code plus difficile à maintenir ou à modifier*

Exemples de code smells :

- *méthodes trop longues*
- *classes trop complexes*
- *duplication de code*
- *variables mal nommées*
- *structures de code compliquées*

Dans le cas de BobApp, les résultats sont :

Projet	Maintainability
Backend (BobApp_Back)	A – 6 code smells
Frontend (BobApp_Front)	A – 5 code smells

Dans le cas du backend, 6 mauvaises pratiques ont été détectées et 5 dans le cas du frontend. Mais dans les deux cas leur impact et la dette technique sont faibles, et le code reste globalement bien structuré et maintenable.

Un tableau avec un exemple d'interprétation plus général de la métrique ***Maintainability*** *est présenté ci-dessous :*

Maintainability	Signification	Niveau
A	Très bonne maintenabilité	Excellent
B	Quelques problèmes mineurs	Bon
C	Plusieurs problèmes	Moyen
D	Code difficile à maintenir	Mauvais
E	Code très difficile à maintenir	Critique

6.4 Security

Cette métrique analyse les failles de sécurité potentielles. Elle repose principalement sur la détection de vulnérabilités dans le code.
Une vulnérabilité est une faiblesse qui peut permettre :

- *un accès non autorisé*
- *la fuite de données*
- *l'exécution de code malveillant*
- *la manipulation des données*

Sonar analyse le code et recherche des patterns connus de failles de sécurité. Il peut détecter des problèmes comme :

- *injections SQL*
- *mauvaise validation des entrées utilisateur*
- *utilisation de bibliothèques non sécurisées*
- *exposition d'informations sensibles*
- *mauvaise gestion de l'authentification*

Dans le cas de BobApp, les résultats sont :

Projet	***Security***
Backend (BobApp_Back)	A – 0 vulnérabilité
Frontend (BobApp_Front)	A – 0 vulnérabilité

Cela indique que dans le cas du backend et du frontend le code ne présente pas de risque de sécurité identifié. Ceci ne garantit pas que l'application est totalement sécurisée, mais simplement qu'aucune faille connue n'a été détectée par l'analyse statique.

Un tableau avec un exemple d'interprétation plus général de la métrique ***Security*** *est présenté ci-dessous :*

Security	***Signification***	***Niveau de sécurité***
A	aucune vulnérabilité détectée	excellent
B	vulnérabilités mineures	bon
C	vulnérabilités modérées	moyen
D	vulnérabilités importantes	mauvais
E	vulnérabilités critiques	très dangereux

6.5 Duplications

Elle mesure le pourcentage de code dupliqué dans un projet. Le code dupliqué correspond à des blocs de code identiques ou très similaires copiés à plusieurs endroits du projet.

La duplication de code est considérée comme une mauvaise pratique car elle complique la maintenance.

Si le même code apparaît dans plusieurs fichiers :

- *une modification doit être faite à plusieurs endroits*
- *il y a plus de risque d'oublier une modification*
- *les bugs peuvent être répétés dans plusieurs parties du code*

Cela rend le projet plus difficile à maintenir et à faire évoluer.
Cette métrique peut être interprétée comme ceci :

Duplications	***Signification***
0 %	aucune duplication détectée
< 3 %	très bon
3 – 10 %	duplication modérée
> 10 %	duplication importante

Dans le cas de BobApp, les résultats sont :

Projet	***Duplications***
Backend (BobApp_Back)	0%
Frontend (BobApp_Front)	0%

Cela signifie qu'aucun bloc de code dupliqué n'a été détecté et que le code est bien factorisé.

6.6 Hotspots Reviewed

La métrique Hotspots Reviewed indique le pourcentage de zones de code sensibles à la sécurité qui ont été examinées par un développeur. Contrairement aux vulnérabilités, les Security Hotspots ne sont pas forcément des failles. Ce sont des portions de code qui nécessitent une vérification manuelle afin de confirmer qu'elles ne présentent pas de risque de sécurité.

SonarCloud signale ces zones car elles peuvent être sensibles.
Exemples :

- *utilisation d'un algorithme de cryptographie*
- *gestion d'un mot de passe*
- *manipulation de données utilisateur*
- *configuration de CORS ou HTTP*

Dans ces cas, SonarCloud ne peut pas déterminer automatiquement si le code est dangereux. Il demande donc à un développeur de reviewer le hotspot.

Dans le cas de BobApp, la métrique Hotspots Reviewed est de 0 % pour le backend et le frontend. Cela signifie que SonarCloud a détecté des zones de code potentiellement sensibles du point de vue de la sécurité, mais qu'aucune d'entre elles n'a encore été examinée manuellement. Ces hotspots constituent un point d'attention important. Des risques de sécurité potentiels n'ont pas encore été validés et doivent être examinés par un développeur pour confirmation.

7 Choix des KPI

Afin d'assurer un niveau de qualité minimal, plusieurs KPI (Key Performance Indicators) ont été définis.
Les principaux KPI utilisés sont :

KPI	Objectif
Coverage	≥ 80 % sur le nouveau code
Bugs	0
Vulnerabilities	0
Duplications	< 3 %

Ces indicateurs permettent de garantir que chaque modification du code respecte un niveau de qualité minimal.

8 Configuration des KPI dans SonarCloud

Les KPI sont configurés via le Quality Gate dans SonarCloud. Le Quality Gate permet de définir des règles de validation du code.

Par exemple :

- *Coverage ≥ 80 % sur le nouveau code*
- *Aucun nouveau bug*
- *Aucune nouvelle vulnérabilité*

Si ces règles ne sont pas respectées, l'analyse Sonar échoue et la Pull Request ne peut pas être validée.

9 Analyse des avis utilisateurs

Les avis utilisateurs permettent d'identifier plusieurs axes d'amélioration.

9.1 Problème 1 : bug sur la publication de blagues

Solution proposée :

- *ajouter des tests sur les fonctionnalités critiques*
- *améliorer la couverture de tests*

9.2 Problème 2 : correction des bugs trop lente

Solution proposée :

- *automatiser les tests avec la CI*
- *détecter les bugs plus rapidement*

9.3 Problème 3 : absence de réponse aux utilisateurs

Solution proposée :

- *mettre en place un système de gestion des issues*
- *améliorer la communication avec les utilisateurs*

10 Conclusion

La mise en place du pipeline CI/CD a permis d'améliorer significativement la gestion et la qualité du projet BobApp.

L'automatisation des différentes étapes du cycle de développement, telles que tests, analyse de code, génération des rapports de couverture et déploiement, a permis de fiabiliser le processus et de réduire les risques d'erreurs humaines. Le projet bénéficie désormais d'un processus de validation continu garantissant une meilleure stabilité du code avant sa mise en production.

Cette démarche s'inscrit dans une logique d'amélioration continue, en renforçant la qualité logicielle, la fiabilité des livraisons et la productivité du développement.

À court terme, des axes d'amélioration ont été identifiés, notamment l'augmentation de la couverture de tests du backend et la correction des problèmes remontés par SonarCloud.

À plus long terme, ce pipeline pourra être enrichi par la mise en place de déploiements continus (CD vers un environnement de production « non implémenté actuellement »), l'ajout de tests end-to-end, ainsi que l'intégration d'outils de monitoring afin de renforcer la fiabilité globale de l'application.